Comparison and Scraping

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Comparison and Scraping

Motivating question:

Which string is bigger: "abacus" or "cat"

Which object is bigger: a balloon, or ten nickels?

```python
numbers = [20, 10, 30]
print(sorted(numbers))

numbers = [Int(20), Int(10), Int(30)]
print(sorted(numbers))
```

```
[10, 20, 30]
Traceback (most recent call last):
File "/Users/rasikabhalerao/2025Fall/2100/Lectures/l25/l25.py", line 34, in <module>
print(sorted(numbers))
      ^^^^^^^^^^^^^^^^
TypeError: '<' not supported between instances of 'Int' and 'Int'
```

Where we're headed...

```python
class Int:
    def __init__(self, number: int) -> None:
        self.number = number

    def __repr__(self) -> str:
        return f"{self.number}"

    def __lt__(self, other: Int) -> bool:
        if not isinstance(other, Int):
            return NotImplemented
        return self.number < other.number

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Int):
            return False
        return self.number == other.number
```

```python
numbers = [
    Int(20),
    Int(10),
    Int(30)
]
print(sorted(numbers))
```

```
[10, 20, 30]
```

Comparable protocol (and no interface)

- `__eq__(self, other: object) -> bool`: equals `==`
- `__ne__(self, other: object) -> bool`: not equals `!=`
- `__lt__(self, other: object) -> bool`: less than `<`
- `__le__(self, other: object) -> bool`: less than or equal to `<=`
- `__gt__(self, other: object) -> bool`: greater than `>`
- `__ge__(self, other: object) -> bool`: greater than or equal to `>=`

Don't need all six

Common: Implement `__eq__()` and one ordering method like `__lt__()`

Exercise: Let's write a class for a Plant. Plant are bigger if they get more sunlight.

```
class Plant:
    def __init__(self) -> None:
        self.sunlight_hours = 0

    def get_sunlight(self) -> None:
        self.sunlight_hours += 1

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Plant):
            return False
        return self.sunlight_hours == other.sunlight_hours

    def __lt__(self, other: object) -> bool:
        if not isinstance(other, Plant):
            return NotImplemented
        return self.sunlight_hours < other.sunlight_hours

plant1 = Plant()
plant2 = Plant()

plant1.get_sunlight()
```

Poll: What goes in the ??? ?

```
class Bouquet:
    """Bouquets are compared by the number of flowers in them"""
    def __init__(self, flowers: list[Flower]):
        self.flowers = flowers

    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Bouquet):
            return False
        return ???

    def __gt__(self, other: object) -> bool:
        if not isinstance(other, Bouquet):
            return NotImplemented
        return ???
```

1. `len(self.flowers) == len(other.flowers)` and `len(self.flowers) < len(other.flowers)`
2. `len(self.flowers) == len(other.flowers)` and `len(self.flowers) > len(other.flowers)`
3. `len(self.flowers) != len(other.flowers)` and `len(self.flowers) < len(other.flowers)`
4. `len(self.flowers) != len(other.flowers)` and `len(self.flowers) > len(other.flowers)`

Poll: What does the comparison protocol do in Python?

1. It lets us define the "length" of an object (the number returned by `len()`)
2. It lets us iterate over the sub-parts of an object (using a `for` loop)
3. It lets us define what it means for an object to be "bigger" or "smaller" than another object
4. It lets us map each object to an int that is used as an index in a hash table

Scraping

Let's save this webpage to our computer: https://en.wikipedia.org/wiki/Web_scraping

(Webpage, Complete)

And let's look at the downloaded HTML file in VSCode.

It contains links to more pages, which contain links to more pages... it's a tree! (Or a graph because it has cycles)

Scraping webpages with BeautifulSoup

Instead of reading a website yourself, you can write code to download the page and extract the data you care about

```
import requests
from bs4 import BeautifulSoup

# 1. Fetch the webpage
response = requests.get("https://example.com")

# 2. Parse the HTML
soup = BeautifulSoup(response.text, "html.parser")

# 3. Find what you want
title = soup.find("h1")
links = soup.find_all("a")

# 4. Use the data!
print(title.text)
  ▶️ "Example Domain"
print(len(links), "links found")
  ▶️ 3 links found
```

Let's look at Homework 7 (Wikipedia Scraping)

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?